

Internal Architecture & Delivery Toolchain

How we actually deliver every customer-facing service — the internal tools, the agents, the integrations, and the per-customer "tweak knobs" that let one delivery toolchain produce bespoke work for every client without writing new code.

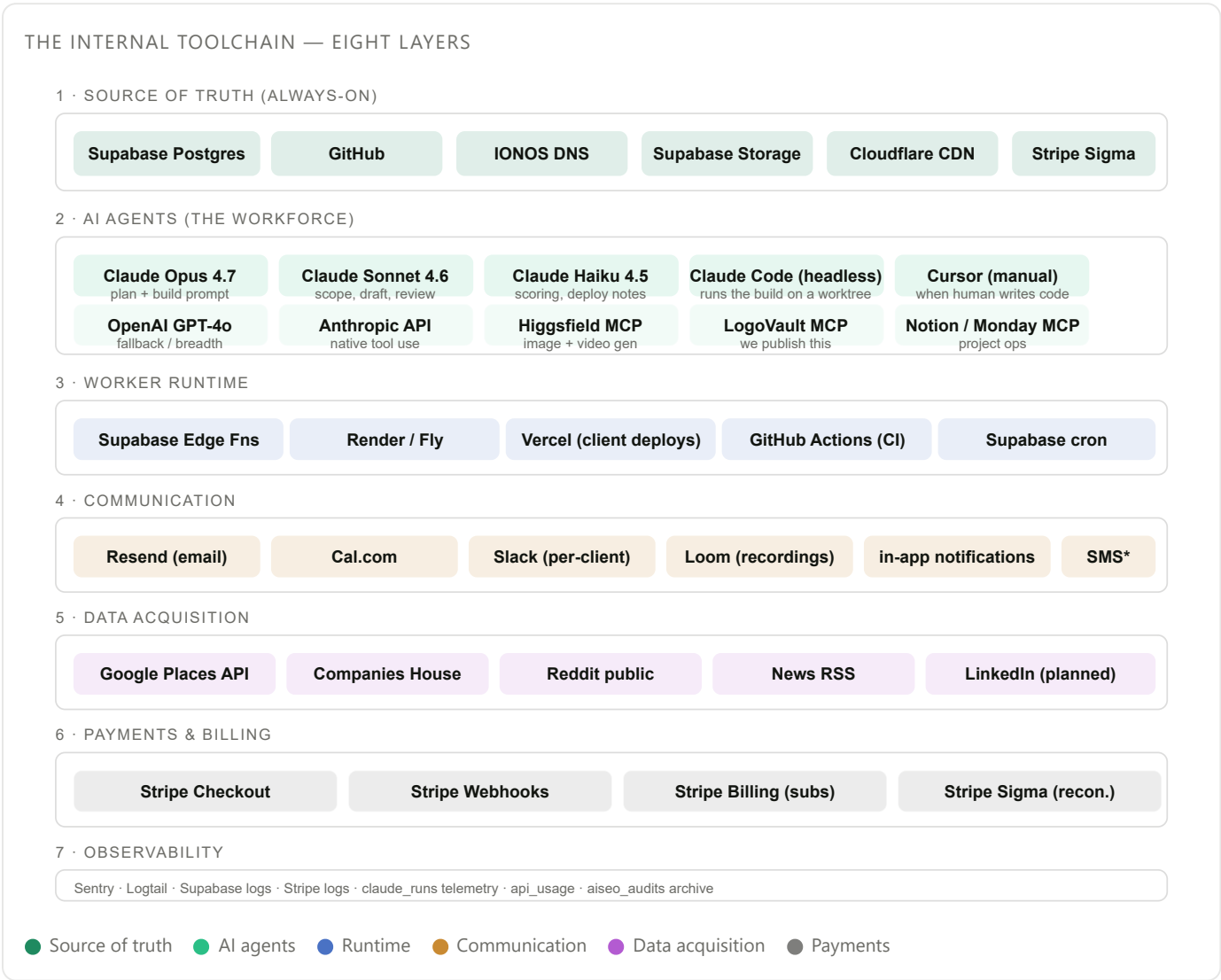
DOCUMENT	DATE	COMPANION	AUDIENCE
INTERNAL OPS DOC V1.0	22 MAY 2026	V3 ROADMAP	YOU + FUTURE TEAM

01	The factory floor — internal services	09	Coaching · 1:1 + courses
02	How customisation stays automated	10	SaaS ops (LogoVault)
03	Master mapping table	11	Lead engine (sold + internal)
04	AI automations · delivery	12	AISEO programme · delivery
05	Websites & landing pages	13	The tweak layer in detail
06	Web apps & SaaS builds	14	MCP & agent inventory
07	Custom internal software	15	Adding a new service
08	Maintenance retainer	16	Cost-to-serve breakdown

01 · THE FACTORY FLOOR

Every internal tool, grouped by job.

This is what we own (or rent) to actually deliver work. Every customer-facing service composes a subset of this list. Nothing custom-built lives outside this map.



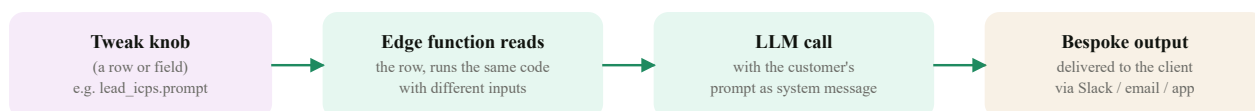
Anything we do that's not on this map = a new entry on this map. We refuse to build a service that requires a tool we don't already operate. Otherwise the surface area expands faster than the senior eng can hold it.

02 · HOW CUSTOMISATION STAYS AUTOMATED

Tweaks live in data, not in code.

Every client gets bespoke output without us writing new code. The magic is that the customisation parameters are *rows in the database*, not new files in a repo.

THE TWEAK LOOP



No code is written. No PR is opened. No deploy is triggered.

A row changes; the next scheduled run produces a different output.

When the client wants something genuinely new, that's when we write code — and that's billed as a build.

Where the tweak knobs live

KNOB	TABLE / FIELD	WHAT IT CONTROLS
Voice prompt	outreach_campaigns.voice_prompt	How outreach drafts sound for this client.
ICP prompt	lead_icps.prompt	How the LLM scores leads against the client's ideal customer.
ICP rules	lead_icps.rules (JSONB)	Structured filters fed to discovery adapters (e.g. places_query, incorporation_within_days).
Send rate	outreach_campaigns.send_rate_per_day	How aggressive the outbound cadence is.
Approval mode	outreach_campaigns.approval_mode	'manual' or 'auto' — flip on when trust is built.
Plan phases	project_plans.phases (JSONB)	Per-project override of a template's standard phases.
Brief context	projects.description + projects.metadata	Passed into every claude_run for this project.
Repo + preview URLs	projects.repo_url, projects.preview_url	Where Claude Code commits + where the client reviews builds.
AISEO questions	aiseo_subscriptions.questions	The 30 buyer-intent queries we ask LLMs about this client.
AISEO engines	aiseo_subscriptions.engines	Which LLM surfaces to monitor.
Notification routing	profiles.notify_email / notify_in_app	Where the client wants to hear from us.
Visibility per doc	project_documents.visible_to_client	What the client sees vs. what stays internal.
Cohort placement	enrollments.cohort_id	Which cohort a learner joins (auto-placed by next-available).
Brand voice (LogoVault private)	logovault_brands.ai_tags	Per-client logo library tagging.

Three rules of the tweak layer

1. **If a tweak takes a row update, do it that way.** No exceptions for "small custom changes" — the next person will ask for "just one more".
2. **If a tweak takes a code change, charge for it.** That's a build, not a customisation. It enters the orchestrator pipeline like any other request.
3. **Every tweak knob has a default.** A new customer is productive on day zero with the platform defaults; tweaks are optional from there.

Every service → every internal tool it touches.

One page. Read horizontally to see what a service needs; read vertically to see how much we depend on a tool.

SERVICE	SUPABASE	CLAUDE API	CLAUDE CODE	STRIPE	RESEND	CAL.COM	SLACK	MCPS	DATA APIS	HOSTING
AI automations	•	•	•	•	•	•	•	•	○	•
Websites & landing	•	•	•	•	•	•	•	•	○	•
Web apps & SaaS builds	•	•	•	•	•	•	•	•	○	•
Custom internal software	•	•	•	•	•	•	•	•	○	•
Maintenance retainer	•	•	•	•	•	—	•	—	—	•
1:1 coaching	•	•	—	•	•	•	—	—	—	—
Cohort courses	•	•	—	•	•	•	•	—	—	—
Business programmes	•	•	○	•	•	•	•	○	—	—
LogoVault SaaS	•	•	—	•	—	—	—	•	—	•
Lead engine (sold)	•	•	—	•	•	○	•	—	•	•
AISEO programme	•	•	•	•	○	○	•	—	○	•

• primary dependency ○ used occasionally / per-project — not used

What this reveals

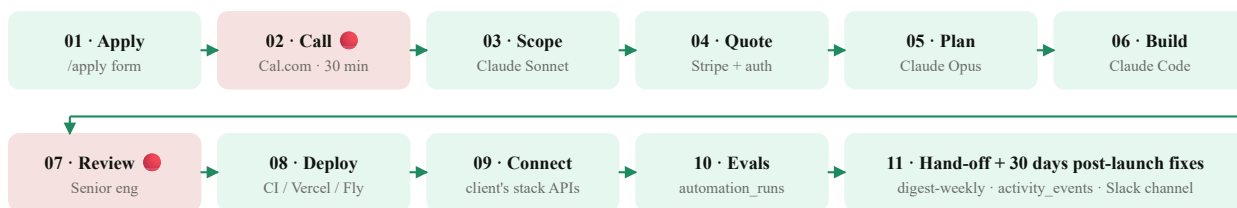
- **Supabase is universal.** Every service writes to it. Losing it = losing the business.
- **Claude API is universal too.** Every service uses LLMs for something. Anthropic outage = degraded service across the board (fallback to OpenAI helps but isn't equivalent for tool use).
- **Claude Code is build-only.** Coaching and courses don't touch it; LogoVault doesn't either. That gives us a smaller blast radius if the worker fails.
- **Slack only matters for delivery services.** Per-client channels are how clients reach the engineer. SaaS and coaching don't need them.
- **MCPs are still nascent for us.** One we publish (LogoVault), several we consume occasionally (Higgsfield, Notion, Monday, Meta Ads). Mostly used as enrichments rather than load-bearing.

04 · AI AUTOMATIONS · DELIVERY TOOLCHAIN

From "we have an inbox triage problem" → live agent on the client's stack.

Flagship service. Highest-touch initial scope; highest-automation delivery. This is the canonical pattern every other build service inherits from.

DELIVERY FLOW



Per-company tweak knobs

- `applications.project_description` — the verbatim brief, fed into every Claude run
- `projects.repo_url` + `projects.metadata` — where Claude Code commits, plus integration secrets (encrypted)

Internal services in play

Supabase (applications, projects, claude_runs, activity_events) · Claude Opus + Sonnet + Haiku · Claude Code headless worker
 Stripe Checkout (£750+ deposit) · Cal.com (kick-off call) · Slack (per-client channel) · Resend (digests)
 GitHub (client repo) · Vercel / Render / Fly (deploy target) · Sentry (errors) · digest-weekly cron (status emails)

The tweak. The customer's brief and stack appear in one place — `applications` + `projects`. The orchestrator pipeline reads the brief and the integration metadata and produces a totally different Claude Code prompt for every client. The code we wrote is constant; the output is bespoke because the input is.

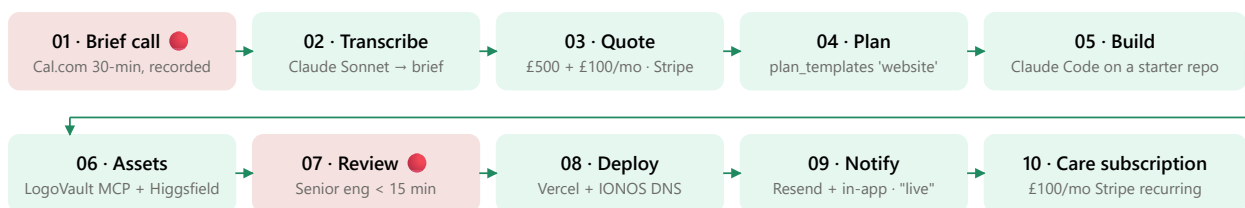
Manual surfaces. The discovery call (stage 02) and the senior code review (stage 07). Both deliberate — the call is the sale, the review is the quality gate. Everything else is scheduled or trigger-driven.

05 · WEBSITES & LANDING PAGES

Brief in the morning, live by the evening.

The most automated build line. A starter site has a single human touchpoint — the brief call — and ships before the call recording's been transcribed.

SAME-DAY-SITE TOOLCHAIN

**Per-company tweak knobs**

- `applications.project_description` — the brief, fed verbatim into the Claude Code prompt
- `applications.company` — drives LogoVault MCP lookup for the customer's logo
- `projects.metadata.brand_colors` — colour palette injected into the Tailwind config of the starter repo
- `projects.metadata.target_audience` — tone of voice + image style for Higgsfield generation
- `project_documents.visible_to_client` — what's in the client doc room vs. our internal scratch

Internal services in play

- **Build:** Claude Code → starter site repo (Next.js + Tailwind) → Vercel deploy.
- **Assets:** LogoVault MCP for logos, Higgsfield MCP for hero imagery and team photos.
- **Hosting:** Vercel (preview) → IONOS (DNS) → live. Care plan keeps it on Vercel with us paying the bill.
- **Care:** `automation_runs` writes "uptime check" rows; nightly cron pings the site, alerts if down.

One starter-site template repo (private) is the seed for every starter site. Claude Code clones, edits, deploys. The whole flow is < 90 minutes of senior-eng time per site — the rest is wall-clock waiting for build/deploy.

06 · WEB APPS & SAAS BUILDS

MVPs in weeks. Same toolchain, more orchestrator passes.

The most code-heavy delivery line. Same internal stack as AI automations, but the Claude Code worker runs many more times — one per slice — and Vercel handles preview deploys per PR.

Differences vs. the standard AI-automation pipeline

PIPELINE ELEMENT	FOR AI AUTOMATION	FOR WEB APP
Plan template	'automation'	'web_app' — auth + billing + multi-tenant phases included
Claude Code runs	1–3	8–20 (one per slice)
Senior reviews	1–3	5–10 (one per slice)
Deploy target	Render / Fly worker	Vercel (PR previews + prod)
Database	None or client's existing	Fresh Supabase project per client
Auth	API-key based	Supabase Auth (email + Google)
Billing	Not in scope	Stripe Checkout wired in week 1
Status surface	activity_events feed	activity_events + per-PR preview URLs

Tweak knobs unique to web app builds

- `projects.preview_url` — Vercel deployment URL, auto-updated per PR.
- `projects.metadata.tenancy` — 'single' or 'multi'. Multi-tenant triggers RLS-by-default code generation.
- `projects.metadata.auth_providers` — array, e.g. ['email', 'google', 'passkey'].
- `projects.metadata.billing_model` — 'subscription', 'metered', or 'one-off'. Drives Stripe wiring in the build prompt.
- `projects.metadata.ai_features` — list of AI features to bake in (RAG, chat, suggest, etc.).

Internal services in play

- **Code:** Claude Code headless worker on Render / Fly — long-running execution per slice.
- **Repo:** private GitHub repo per client; we keep admin until client takes over post-handoff.
- **Hosting:** Vercel for the front-end, Supabase for the back-end. Both transferable to client account at handoff.
- **Observability:** Sentry (errors), Supabase logs, `claude_runs` (LLM cost per slice).
- **Comms:** Slack channel per client, per-PR preview URLs auto-posted, weekly digest email via `digest-weekly`.

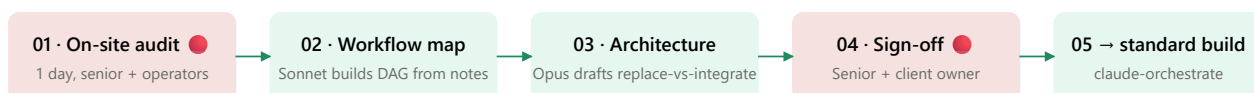
The tweak that does the most work. `projects.metadata.ai_features` — listing things like ['rag-over-docs', 'inbox-triage', 'recommendation-engine'] — gets joined to template fragments in the planner. Each named feature is a known prompt module we've already shipped before; the LLM stitches them into the project's plan.

07 · CUSTOM INTERNAL SOFTWARE

The spreadsheet replacement business.

Same orchestrator pipeline. The difference is the discovery phase — we sit with operators on-site and use a structured workflow-mapping LLM session to produce the brief.

DISCOVERY LAYER (THE ONLY STAGE THAT DIFFERS)



Per-company tweak knobs

- `projects.metadata.workflow_dag` — JSON of the audited workflow; fed into every Claude Code build prompt for the project
- `projects.metadata.existing_tools` — what we integrate with vs. what we replace
- `projects.metadata.operator_personas` — who uses what UI; drives role-based access design

Internal services unique to this line

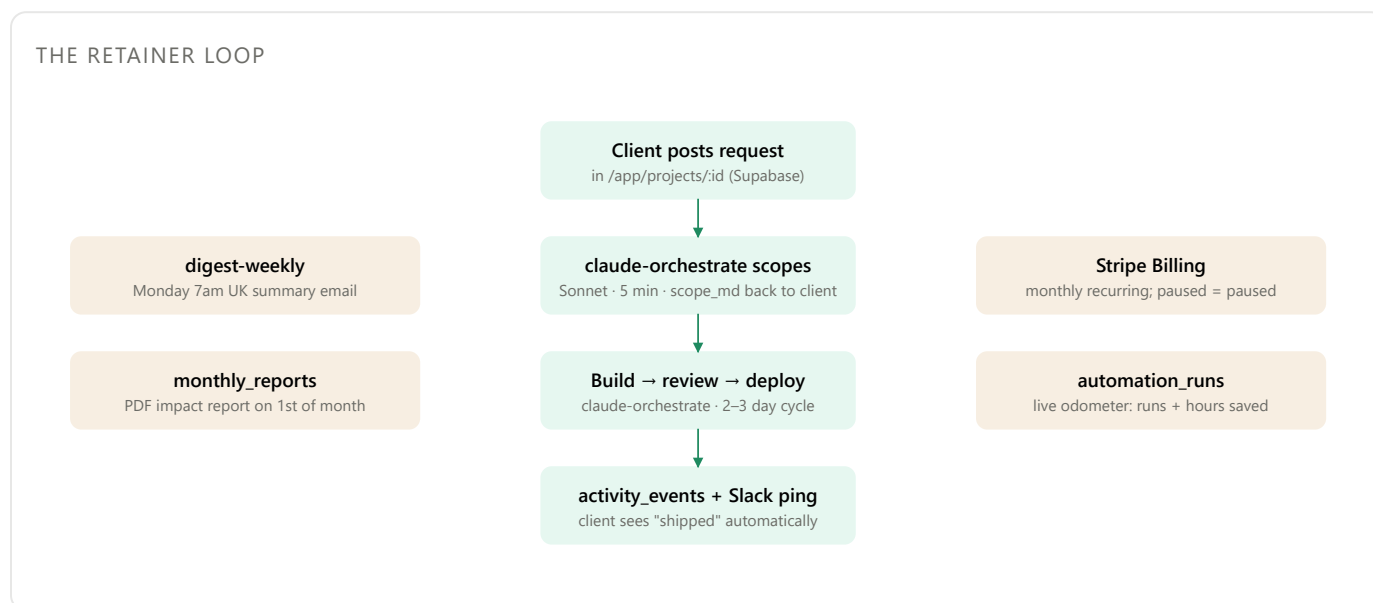
- **Workflow capture toolkit (planned Q3 2026):** a Notion template + LLM session script. Senior eng captures workflows in Notion using a structured template; an edge function extracts the DAG into JSONB on the project record.
- **Retool / refine.dev as fallback:** for very lightweight admin UIs, we use Retool with a generated config. For anything serious, we generate a custom React admin under `/app/admin/*` in the client's repo.
- **Slack escalation bot (planned):** when an operator marks a workflow as "stuck", the bot writes a `feature_request` and pings the engineer.

Manual surfaces. The on-site audit (irreducible — we need eyes on the actual ops floor) and the architecture sign-off (the moment the client commits to the build). Everything after stage 04 is the same automated pipeline as AI automations.

08 · MAINTENANCE RETAINER

The flywheel. Every retainer is a request loop running indefinitely.

Once a client is live, this is the steady-state interface. Most of our recurring revenue. Almost zero net-new infrastructure — it reuses every system already on this map.



Tweak knobs (these are the retainer "configuration")

- `subscriptions.plan` — 'lite' / 'standard' / 'heavy' — drives senior-eng hours/month + response SLA.
- `projects.health_status` — 'healthy' / 'degraded' / 'down' — auto-updated by uptime ping; drives Slack alert priority.
- `feature_requests.priority` — client-set; surfaces in order in the dev board.
- `projects.metadata.on_call` — boolean. Premium retainers get out-of-hours response.
- `profiles.notify_email` / `notify_in_app` — per-user preference, no separate config to maintain.

The retainer is just the platform running on autopilot for one specific client. No new internal tools, no new templates. Tools we built for delivery become the daily interface for the relationship.

09 · COACHING & COURSES

The education stack — same database, very different toolchain.

Almost no Claude Code usage. Much more Cal.com, Stripe, Slack, Resend. The learner lifecycle uses the same Supabase schema but doesn't touch the build pipeline.

1:1 COACHING TOOLCHAIN



COHORT COURSE TOOLCHAIN



Per-learner tweak knobs

- `enrollments.capstone_url` — what they submitted · `enrollments.capstone_grade` — what they earned
- `profiles.timezone` — drives session-time formatting in emails · `enrollments.notes` — anything bespoke for that learner

Business programmes (£1.5k / £3.5k / £6.5k)

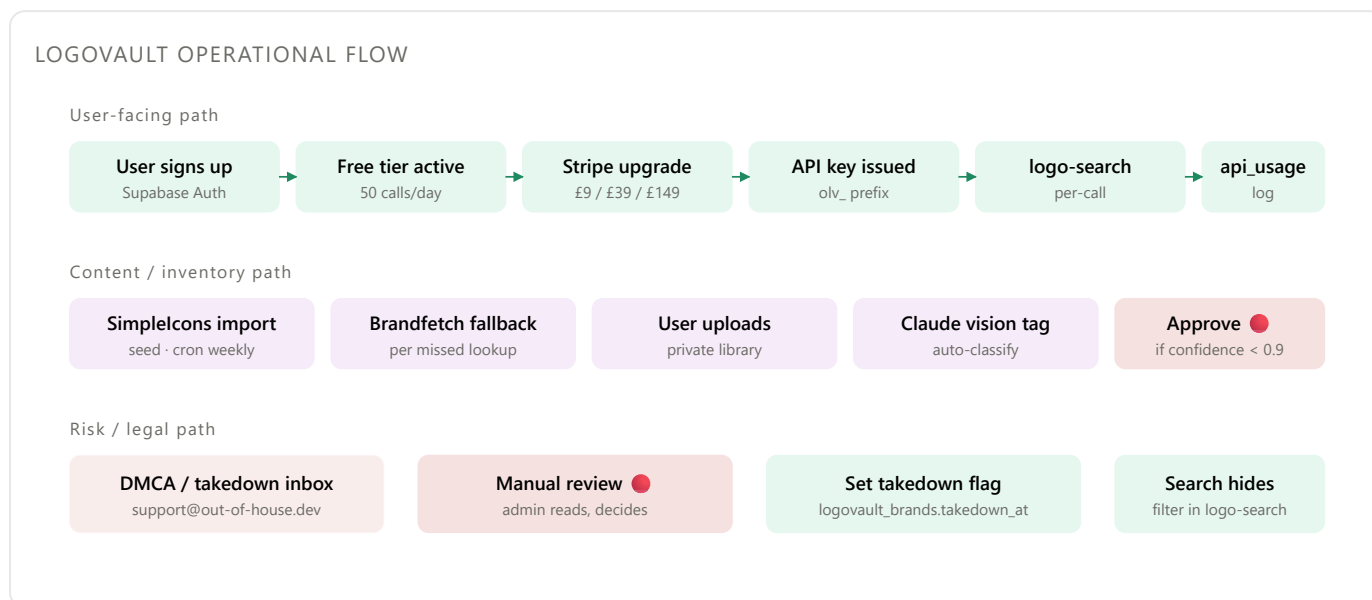
Same as cohort course pipeline + a workflow audit phase at the start (reusing the internal-software audit toolkit) + Slack escalation for the client team. We sometimes spin up small Claude automations as part of the programme; those go through the standard claude-orchestrate pipeline.

The shared insight. Coaching and courses don't need Claude Code at all (almost zero builds happen during a cohort). They lean heavily on Cal.com + Stripe + Slack + Resend. That makes the coaching line operationally simple — much fewer moving parts than a build.

10 · SAAS OPERATIONS (LOGOVAULT AS EXEMPLAR)

SaaS = product running itself, with us watching.

The most "set and forget" line. Once seeded, the only manual surfaces are DMCA takedowns and onboarding new brands above the AI tagger's confidence threshold.



Internal services in play

- **Inventory build:** Simpleicons (CC0) seed + Brandfetch on-demand fallback (cached) + user uploads with Claude vision tagging.
- **Search:** Postgres tsvector index on `logovault_brands.search_tsv` — no Elasticsearch.
- **Delivery:** Cloudflare CDN in front of Supabase Storage for assets.
- **Billing:** Stripe Billing subscription per plan tier. `api_usage` meters per call; overages billed monthly.
- **Distribution:** direct UI + REST API + an MCP server we publish so Claude Code / Cursor users can call LogoVault as a tool.

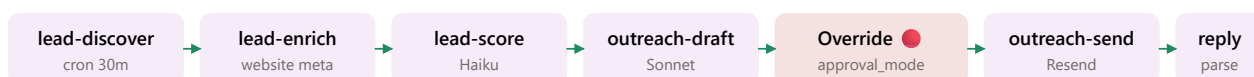
The customer tweak. A LogoVault user's "tweak" is uploading their own brand library, which gets indexed alongside the public catalogue. Per-API-key scope means their private logos stay private without us writing any access-control code beyond standard RLS on `logovault_brands`.

11 · LEAD ENGINE (SOLD + INTERNAL)

The same pipeline serves us and serves clients — partitioned by RLS.

We run our own ICP through the engine to feed our own pipeline. Clients run their own ICPs through the same engine. Same code, same cron, different rows.

ENGINE PIPELINE (CLIENTS + US, ON THE SAME RAILS)



The tweak layer — every client/our ICP is one row

lead_icps row (one per client):

- **prompt** — LLM scoring instruction for this client
- **rules** — JSONB filters passed to discovery adapters
- **channels** — email / linkedin / x

outreach_campaigns row (one per ICP per channel):

- **voice_prompt** — write like the client's founder
 - **send_rate_per_day** — how aggressive · **approval_mode** — manual or auto
- All leads, outreach_messages, lead_signals rows scope to client_id · RLS handles isolation

Why one engine serves everyone

- Every lead row has a `client_id`. RLS prevents one client from seeing another's leads — the same way we isolate projects.
- Adapters (places, companies_house, reddit) read from `lead_sources` which is global. Adding a new source benefits every client.
- Per-client ICP + voice are the only "code-shaped" knobs. Both are LLM prompts stored as text.
- When a client wants something genuinely outside the engine (e.g. integrate their CRM), that's a paid AI-automation build on top of the engine, not a fork of it.

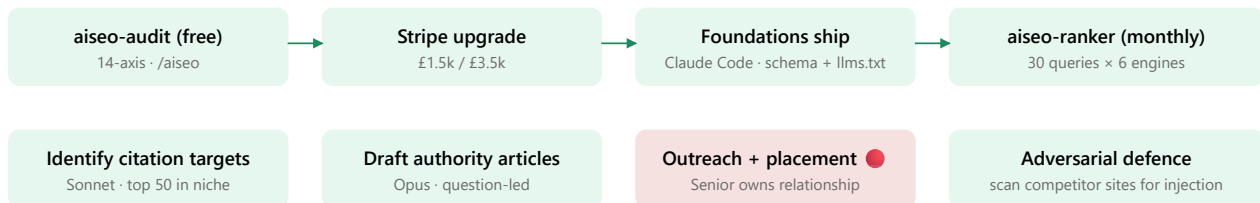
The tweak that makes us look bespoke. `voice_prompt` + `lead_icps.prompt` are 2–10 paragraphs of natural-language English. Per client, that's where the bespoke work lives. Our code reads those two paragraphs and produces wildly different outputs for every customer — without ever being modified.

12 · AISEO PROGRAMME · DELIVERY

Audit → schema → authority → measure → tune.

The most "engineer-as-craftsman" line. Foundations are automated; authority work needs human judgement (we draft with AI, decide with humans).

AISEO RETAINER PIPELINE

**Per-company tweak knobs**

- `aiseo_subscriptions.questions` — the 30 buyer-intent queries we ask LLMs for this brand
- `aiseo_subscriptions.engines` — which LLM surfaces to monitor (default: all 6)
- `aiseo_subscriptions.tier` — drives number of citation placements + articles per month
- `projects.metadata.niche` — fed into citation-target identification + authority article topics
- `projects.metadata.voice_guide` — same shape as outreach voice_prompt; defines how draft articles read
- `projects.metadata.competitors` — adversarial-defence target list

Internal services in play

- **Audit:** `aiseo-audit` edge function (deterministic, no LLM cost on the audit itself).
- **Foundations build:** the standard Claude Code orchestrator — schema + llms.txt + page architecture changes are just normal build slices.
- **Monthly ranking pull:** `aiseo-ranker` (planned Q3 2026) — hits each of the 6 LLM engines via their respective APIs, captures presence/rank/citation.
- **Authority drafting:** Opus generates the long-form article in our voice; senior eng edits before publication.
- **Adversarial defence:** cron scans competitor sites for indirect prompt-injection attempts; alerts to client Slack.

The red box. Outreach for citation placement (stage 3 of the authority row). LLMs can draft the pitch; humans own the relationship. We won't automate the "hey, would you link to this" outreach because relationship-based placements aren't credible when AI-sent.

13 · THE TWEAK LAYER IN DETAIL

Where bespoke meets automated.

For every service, the customisation surface is a handful of editable rows. This page catalogues exactly what's editable per customer, who edits it, and how often.

SERVICE	PRIMARY TWEAK ROW(S)	WHO EDITS	HOW OFTEN
AI automations	<code>projects.description</code> · <code>projects.metadata.ai_features</code> · <code>projects.metadata.integration_secrets</code>	Admin (us)	Once at intake, occasionally as scope evolves
Websites	<code>applications.project_description</code> · <code>projects.metadata.brand_colors</code> · <code>projects.metadata.target_audience</code>	Admin	Once at intake
Web apps / SaaS builds	<code>projects.metadata.tenancy</code> · <code>auth_providers</code> · <code>billing_model</code> · <code>ai_features</code>	Admin	Once at intake; one or two changes per build
Internal software	<code>projects.metadata.workflow_dag</code> · <code>existing_tools</code> · <code>operator_personas</code>	Admin (built during audit)	Once at audit, refined during build
Maintenance	<code>subscriptions.plan</code> · <code>projects.health_status</code> · <code>profiles.notify_*</code>	Admin + client (notify prefs)	Monthly review
1:1 coaching	<code>coaching_bookings.topic</code> · <code>profiles.timezone</code>	Client	Per booking
Cohort courses	<code>enrollments.capstone_url</code> · <code>capstone_grade</code> · <code>enrollments.notes</code>	Senior eng (grade) + learner (capstone)	Once per cohort cycle
Business programmes	<code>projects.metadata.workflow_dag</code> · <code>enrollments.notes</code> · per-programme automation rows	Admin	Weekly through cohort
LogoVault	<code>logovault_brands.*</code> (user uploads) · <code>api_keys.name</code> · <code>api_keys.scopes</code>	User (self-serve)	Per upload
Lead engine	<code>lead_icps.prompt</code> · <code>lead_icps.rules</code> · <code>outreach_campaigns.voice_prompt</code> · <code>approval_mode</code>	Admin (us) + client (review on review)	Initial setup; monthly tuning
AISEO	<code>aiseo_subscriptions.questions</code> · <code>engines</code> · <code>projects.metadata.niche</code> · <code>voice_guide</code> · <code>competitors</code>	Admin (us)	Quarterly

What if a client wants something that's NOT a tweak knob?

It enters the standard build pipeline. The client posts a `feature_request`; `claude-orchestrate` scopes it; senior eng approves; build runs. Result: a new column, a new table, or a new edge function — committed to the platform and available to every client from then on.

This is how we compound. Every bespoke ask becomes a tweak knob the next client can use without a build. Year 1 the menu of knobs is short; year 3 it's the deepest in the market.

14 · MCP & AGENT INVENTORY

What we publish, what we consume, what we plan.

MCP servers turn our internal tools into composable building blocks for any LLM. The list is short today; it's a deliberate roadmap, not an accident.

MCP servers we publish

SERVER	STATUS	WHY IT EXISTS	WHO USES IT
LogoVault MCP	Beta	Search logos from inside Claude / Cursor without leaving the editor.	Customers + us
Prompt Locker MCP	Planned Q3 2026	Recall vetted prompts from inside any LLM-enabled editor.	Customers + us
Plan Library MCP	Planned Q4 2026	Surface our plan_templates inside Claude Code so clients can read our standard plans.	Customers + us
Audit MCP	Planned Q1 2027	Expose AISEO audit + AISEO ranking pulls as Claude tools so any client agent can self-audit.	Customers

MCP servers we consume

SERVER	USE CASE
Higgsfield MCP	Hero imagery + team photos + ad creative imagery for delivery work.
Notion MCP	Workflow capture (internal software audits) + course content storage.
Monday MCP	Project + sprint tracking when a client uses Monday and we mirror to projects.
Meta Ads MCP	Ads-push pipeline — generated creatives uploaded to Meta Marketing API.
GitHub MCP (built-in)	PR opens, branch creation, diff inspection.

Internal agent roles (what each Claude model does)

ROLE	MODEL	WHERE IT LIVES
Scoper	Claude Sonnet 4.6	claude-orchestrate stage 01
Planner	Claude Opus 4.7	claude-orchestrate stage 02
Build-prompt writer	Claude Opus 4.7	claude-orchestrate stage 03
Builder	Claude Code (headless)	Separate worker repo (planned Render/Fly)
Reviewer	Claude Sonnet 4.6 +  senior	claude-orchestrate stage 05
Deployer	Claude Haiku 4.5	claude-orchestrate stage 06
Lead scorer	Claude Haiku 4.5	lead-score
Outreach drafter	Claude Sonnet 4.6	outreach-draft
Digest writer	Claude Sonnet 4.6	digest-weekly
Ad creative writer	Claude Sonnet 4.6	ads-generate

ROLE	MODEL	WHERE IT LIVES
Capstone grader	Claude Sonnet 4.6 +  senior	Course grading flow
Citation researcher	Claude Sonnet 4.6 (planned)	AISEO authority pipeline
Authority writer	Claude Opus 4.7 (planned)	AISEO authority pipeline
Adversarial scanner	Claude Haiku 4.5 (planned)	AISEO adversarial defence

Every role above has a system prompt stored in code (or in `plan_templates.claude_code_handoff` for the building part). When we say "tune the model", we mean edit one system prompt — not retrain anything.

15 · HOW A NEW SERVICE GETS ADDED TO THE PLATFORM

The standard playbook.

The platform's value is that adding service line #9, #10, #11 takes a week instead of a month. This is the checklist.

1. **Decide if it's a service, a SaaS, or a programme.** Service = bespoke work using existing internal tools. SaaS = self-serve subscription. Programme = cohort-based teaching. Each has a different schema home.
2. **Write the marketing page first.** A new entry in `src/data/services.js` (delivery service) or `src/data/programmes.js` (programme) or `src/data/saasApps.js` (SaaS). The site dropdown auto-updates.
3. **Identify the new tweak knobs.** What columns / JSONB fields will the customer's customisation live in? Add them to the relevant tables in a fresh migration.
4. **Decide which existing edge functions can serve it.** 90% of the time the answer is "claude-orchestrate plus the new tweak knobs". Resist writing a new edge function.
5. **Add a Stripe product.** One product per pricing tier; metadata links back to the slug in the data file.
6. **Write the plan template.** Add a row to `plan_templates` with phases + a Claude Code handoff brief.
7. **Wire Cal.com if there's a call.** A sub-path on cal.com per service surface.
8. **Add the page-level analytics events.** Drop into `activity_events` for in-platform; nothing third-party.
9. **Ship the marketing copy.** Apply for AISEO inside the platform — same way clients do — to catch our own positioning.
10. **Add a row to this document.** Update the master mapping table on page 03 + the per-service spread.

What this looks like in practice

STEP	FILES TOUCHED FOR "AISEO" ADDITION
1. Marketing data	<code>src/data/aiseo.js</code>
2. Marketing page	<code>src/pages/AISEO.js</code>
3. Route + nav	<code>src/App.js</code> + <code>src/Header.js</code>
4. Tweak-knob tables	<code>supabase/migrations/005_v3_business_expansion.sql</code> (aiseo_audits, aiseo_subscriptions, aiseo_rankings)
5. Edge function (new behaviour)	<code>supabase/functions/aiseo-audit/</code>
6. Stripe products	(external, manual config — checklist item)
7. CSS	<code>src/styles/v3.css</code>
8. PDF entries	This doc + the v3 roadmap

Total time for the AISEO addition in this session: ~30 minutes of design + ~90 minutes of build. The platform's design is what makes that possible — every other service "shape" had been encoded already.

16 · COST TO SERVE, PER SERVICE

How much it costs us in variable spend to deliver one unit.

Knowing this is the difference between pricing for capacity and pricing for margin.

SERVICE / UNIT	LLM COST	EMAIL / SMS	HOSTING / API	SENIOR ENG TIME	TOTAL VARIABLE £
1 AI automation build (£3k average)	£12	£0.20	£3	6 hrs @ £100 = £600	~£615
1 starter website (£500)	£3	£0.10	£2	1.5 hrs = £150	~£155
1 web app MVP (£8k average)	£35	£0.40	£10	24 hrs = £2,400	~£2,445
1 month maintenance retainer (£2k)	£18	£0.50	£8	6 hrs = £600	~£625
1 coaching hour (£100)	£0.40	£0.10	£0.10	1 hr = £100	~£100.60
1 dev course enrolment (£395 @ 3w)	£3	£0.15	£0.50	12 hrs / cohort of 16 = £75	~£79
1 LogoVault Studio sub (£39/mo)	£0	£0.05	£2.70	~0	~£2.75
1 lead engine month (£1,000/mo)	£40	£3	£25	4 hrs = £400	~£468
1 AISEO Authority month (£1,500/mo)	£28	£0.30	£18	6 hrs = £600	~£646

What this tells us

- **Senior eng time dominates every cost line.** The platform isn't expensive to run; the engineer behind it is the cost.
- **SaaS is the only line where variable cost is near zero.** That's why SaaS revenue is structurally different — every new sub is almost-pure margin once the product exists.
- **LLM cost is < 5% of total cost on every line.** Provider price hikes are absorbable; engineer time hikes are not.
- **The way to scale is to drive senior-hours-per-unit down.** Either by automating more (claude-orchestrate auto-merge) or by leveraging the dev bench (more engineers at a similar quality bar).

The point of the platform is to make each senior-eng hour produce more revenue than the year before. The map in this document is how we do it — every system reduces what a human has to do per customer outcome.